

Finite Automata

- **Deterministic Finite Automaton (DFA)**
- **Non-Deterministic Finite Automaton (NFA)**
- **Equivalence of DFA and NFA**

Deterministic Finite Automaton (DFA)

Deterministic Finite Automaton (DFA)

A **Deterministic Finite Automaton (DFA)** is a quintuple

$$A = (Q, \Sigma, \delta, q_0, F)$$

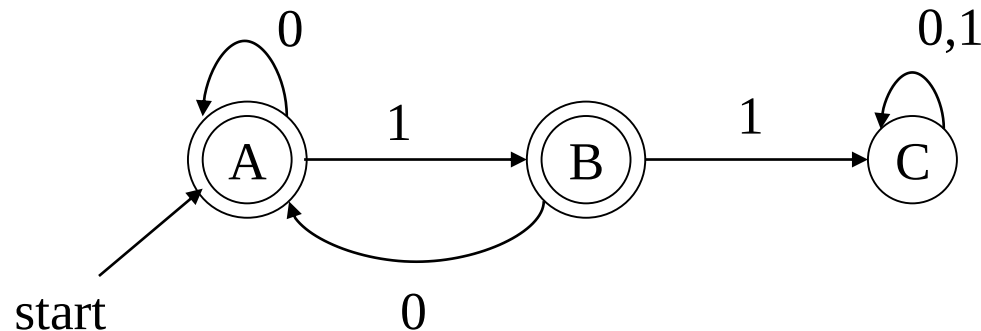
1. Q is a **finite set of states**
 2. Σ is a **finite set of symbols** (alphabet)
 3. Delta (δ) is a **transition function** $(q,a) \rightarrow p$
 4. q_0 is the **start state** ($q_0 \in Q$)
 5. F is a set of **final (accepting) states** ($F \subseteq Q$)
-
- Transition function takes two arguments: a state and an input symbol.
 - $\delta(q, a)$ = the state that the DFA goes to when it is in state q and input a is received.

Graph Representation of DFA

- Nodes = states.
- Arcs represent transition function.
 - Arc from state p to state q labeled by all those input symbols that have transitions from p to q .
- Arrow labeled “Start” to the start state.
- Final states indicated by double circles.

Graph Representation of a DFA: Example

A DFA: **Accepts all strings without two consecutive 1's.**



DFA = (Q, Σ , δ , q_0 , F)

– **Q = {A,B,C}**

$\Sigma = \{0,1\}$

$q_0 = A$

F = {A,B}

- **States:**

- State A: previous string is OKAY (does not contain 11), and it does not end in 1.
- State B: previous string is OKAY (does not contain 11), and it ends in 1.
- State C: previous string contains two consecutive 1's (it is NOT OKAY).

Alternative Representation: Transition Table

Columns = input symbols

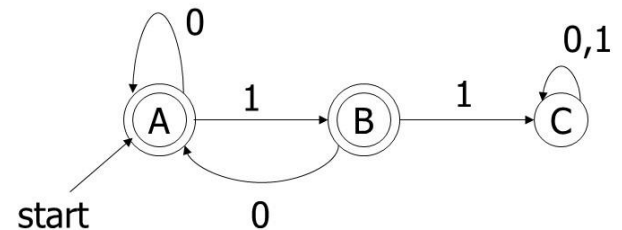
Final states starred

Arrow for
start state

→ * A
* B
C

Rows = states

	0	1
A	A	B
B	A	C
C	C	C



Strings Accepted by a DFA

- An DFA accepts a string $w = a_1a_2 \dots a_n$ if its path in the transition diagram that
 1. Begins at the start state
 2. Ends at an accepting state

- This DFA accepts input: 010001

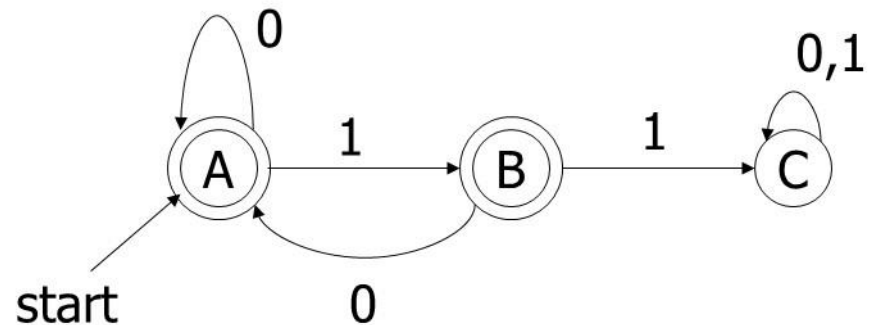
$A \xRightarrow{0} A \xRightarrow{1} B \xRightarrow{0} A \xRightarrow{0} A \xRightarrow{0} A \xRightarrow{1} B$

- This DFA rejects input: 011001

$A \xRightarrow{0} A \xRightarrow{1} B \xRightarrow{1} C \xRightarrow{0} C \xRightarrow{0} C \xRightarrow{1} C$

- This DFA accepts input: 0000

$A \xRightarrow{0} A \xRightarrow{0} A \xRightarrow{0} A \xRightarrow{0} A$



Extended Delta Function – Delta Hat $\hat{\delta}$

- The transition function δ can be extended to **extended delta function $\hat{\delta}$** that operates on **states** and **strings** (as opposed to states and symbols).
- **Extended delta function $\hat{\delta}$** can be defined induction on length of string.

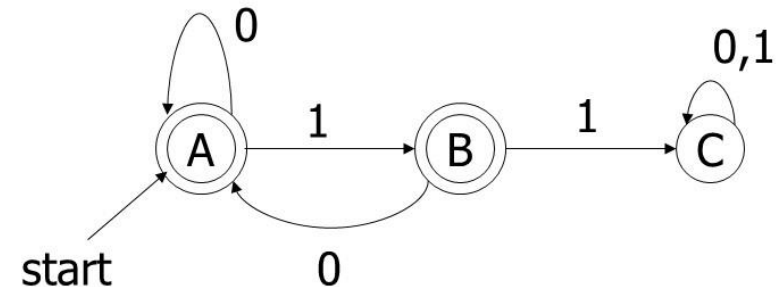
Basis: $\hat{\delta}(q, \epsilon) = q$ when the string is the empty string ϵ

Induction: $\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$ when the string is a non-empty string xa
where a is an input symbol and x is a string

Extended Delta Function – Delta Hat $\hat{\delta}$: Example

- Computing $\hat{\delta}(A, 0100)$
 - Computes $\hat{\delta}$ for all prefixes of 0100

- $\hat{\delta}(A, \epsilon) = A$
 - $\hat{\delta}(A, 0) = \delta(\hat{\delta}(A, \epsilon), 0) = \delta(A, 0) = A$
 - $\hat{\delta}(A, 01) = \delta(\hat{\delta}(A, 0), 1) = \delta(A, 1) = B$
 - $\hat{\delta}(A, 010) = \delta(\hat{\delta}(A, 01), 0) = \delta(B, 0) = A$
 - $\hat{\delta}(A, 0100) = \delta(\hat{\delta}(A, 010), 0) = \delta(A, 0) = A$
-
- Since $\hat{\delta}(A, 0100) = A$ and A is a final state, the string 0100 is accepted by this DFA.



Language Accepted by a DFA

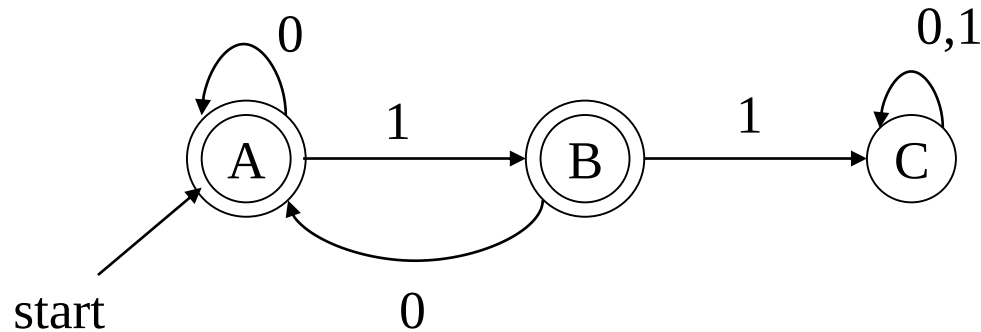
- Informally, the language accepted by a DFA A is the set of all strings that are recognized by A .

- **Formally, the language accepted by a DFA A is $L(A)$ such that**

$$L(A) = \{ w \mid \hat{\delta}(q_0, w) \in F \} \quad \text{where } q_0 \text{ is the starting state of } A \text{ and } F \text{ is the final states of } A$$

- **Languages accepted by DFAs are called as regular languages.**
 - Every DFA accepts a **regular language**, and
 - For every **regular language** there is a DFA that accepts it

Language Accepted by a DFA

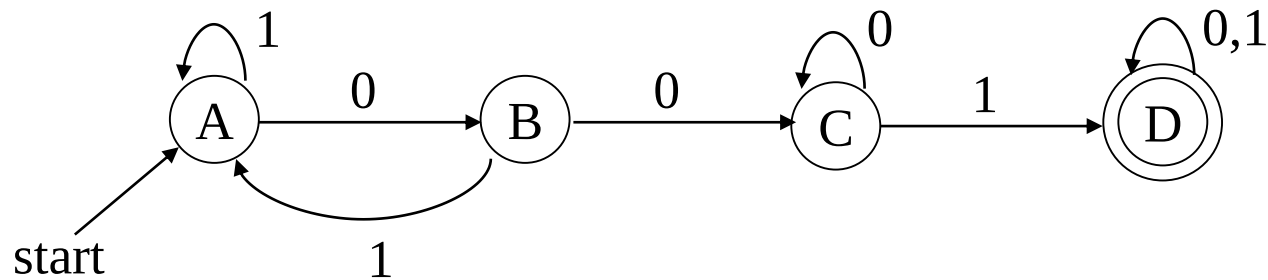


- This DFA accepts all strings of 0's and 1's without two consecutive 1's.
- Formally,

$$L(A) = \{ w \mid w \text{ is in } \{0,1\}^* \text{ and } w \text{ does not have two consecutive 1's} \}$$

DFA Examples

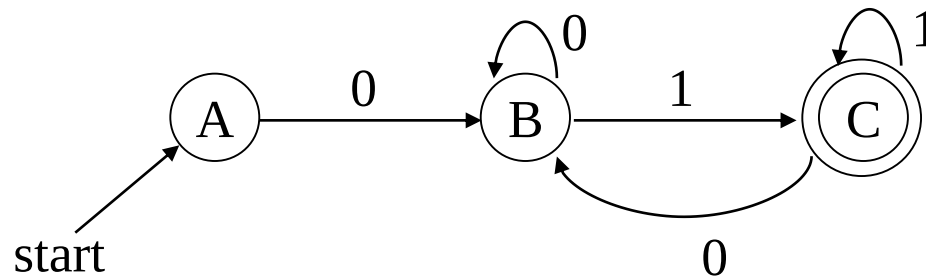
- A DFA accepting all strings of 0's and 1's containing 001.



- What do states represent?
 - A: (empty string) OR (strings do not contain 001 and end in 1)
 - B: (string 0) OR (strings do not contain 001 and end in 10)
 - C: strings do not contain 001 and end in 00
 - D: strings contain 001

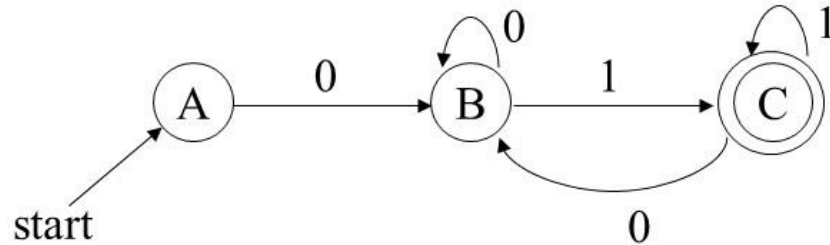
DFA Examples

- A DFA accepting all strings of 0's and 1's which start with 0 and end in 1.

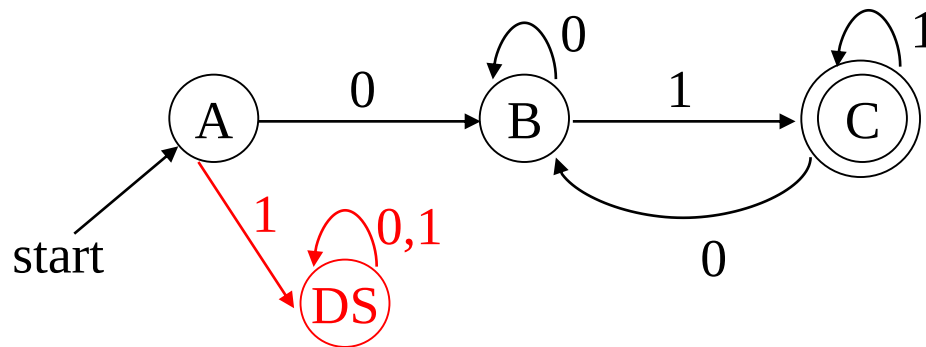


- What do states represent?
 - A: empty string
 - B: strings start with 0 and end in 0
 - C: strings start with 0 and end in 1

DFA Examples: Missing Arcs

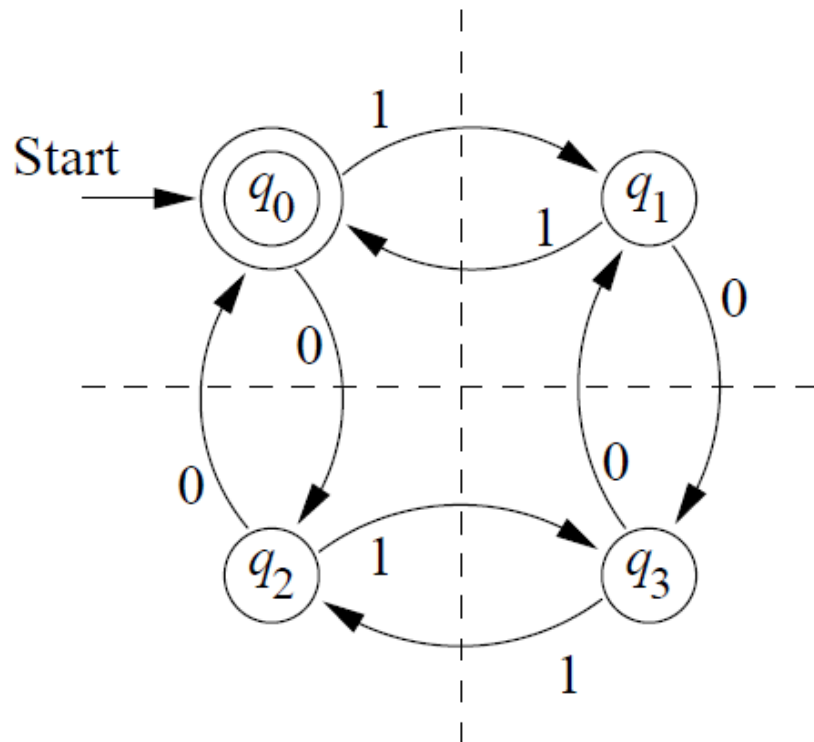


- State A does not have any arc with 1.
 - What happens when symbol 1 comes when we are in state A?
- We assume that all missing arcs go to a **death state DS**, **DS goes to itself for all symbols** and **DS is a non-accepting state**.



DFA Examples

- A DFA accepting all and only strings with an even number of 0's and an even number of 1's



What do states represent?

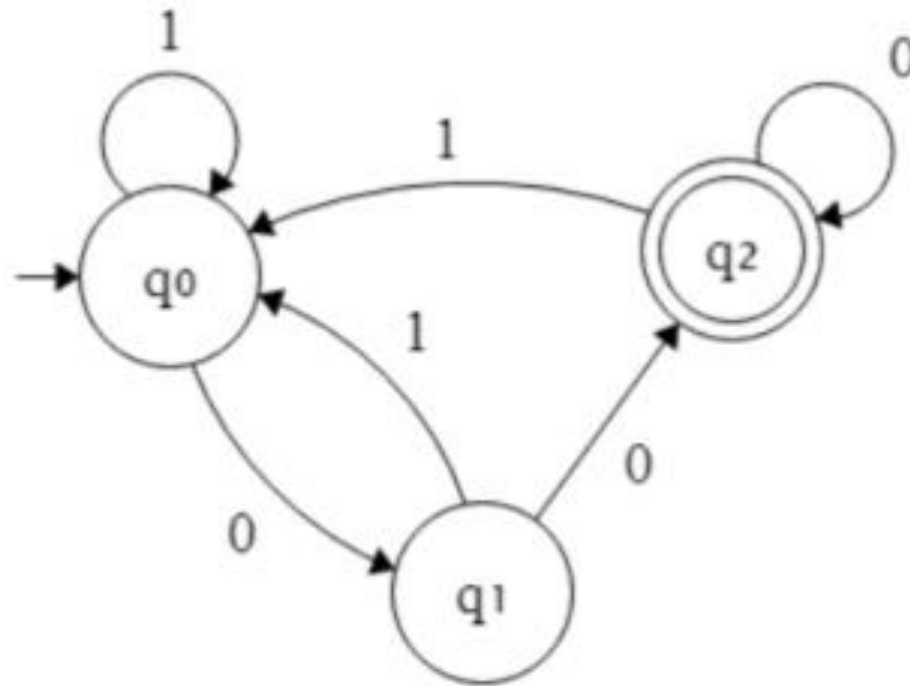
- q_0 : strings with an even number of 0's and an even number of 1's
- q_1 : strings with an even number of 0's and an odd number of 1's
- q_2 : strings with an odd number of 0's and an even number of 1's
- q_3 : strings with an odd number of 0's and an odd number of 1's

DFA Examples: Questions?

- Give DFA's accepting the following languages over the alphabet $\{0,1\}$.
 1. The set of all strings ending in 00.
 2. The set of all strings. i.e. $\{0,1\}^*$
 3. The set of all non-empty strings. i.e. $\{0,1\}^+$
 4. The empty language. i.e. $\{\}$
 5. The language that contains only the empty string. i.e. the set $\{\epsilon\}$
 6. The language $\{0^n 1^k \mid n \geq 1 \text{ and } k \geq 1\}$
 7. The strings whose second characters from the right end are 1.
 8. The strings whose third characters from the right end are 1.

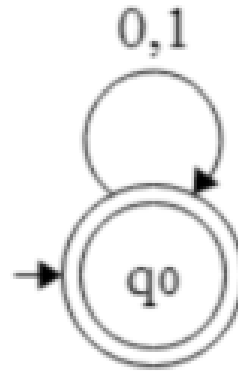
DFA Examples: Questions?

Language over alphabet $\{0,1\}$: The set of all strings ending in 00



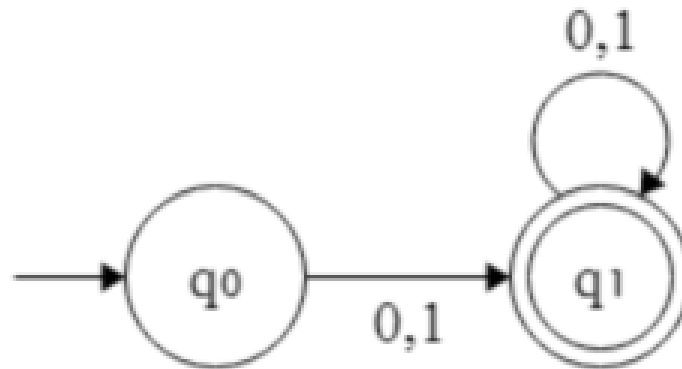
DFA Examples: Questions?

Language over alphabet $\{0,1\}$: The set of all strings. i.e. $\{0,1\}^*$



DFA Examples: Questions?

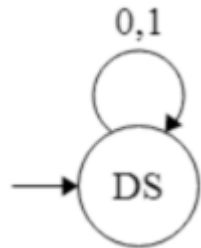
Language over alphabet $\{0,1\}$: The set of all non-empty strings. i.e. $\{0,1\}^+$



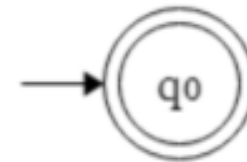
DFA Examples: Questions?

Languages over alphabet $\{0,1\}$

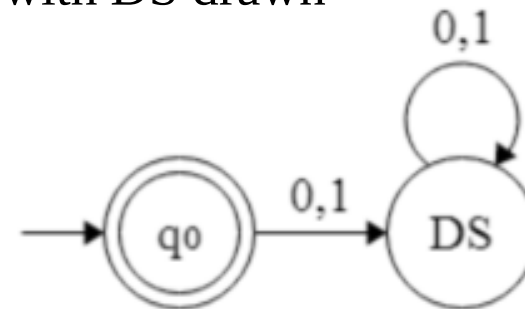
The empty language. i.e. $\{\}$



The language that contains only the empty string. i.e. the set $\{\epsilon\}$

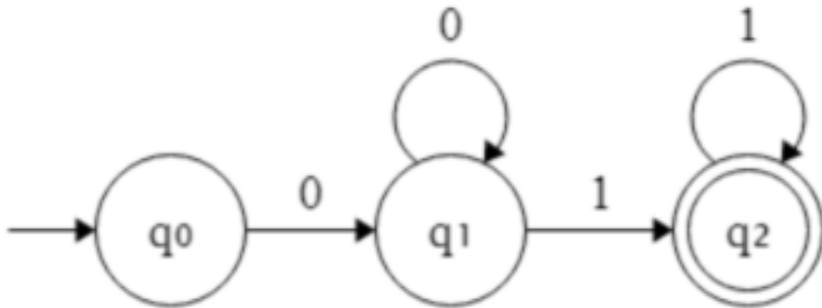


with DS drawn

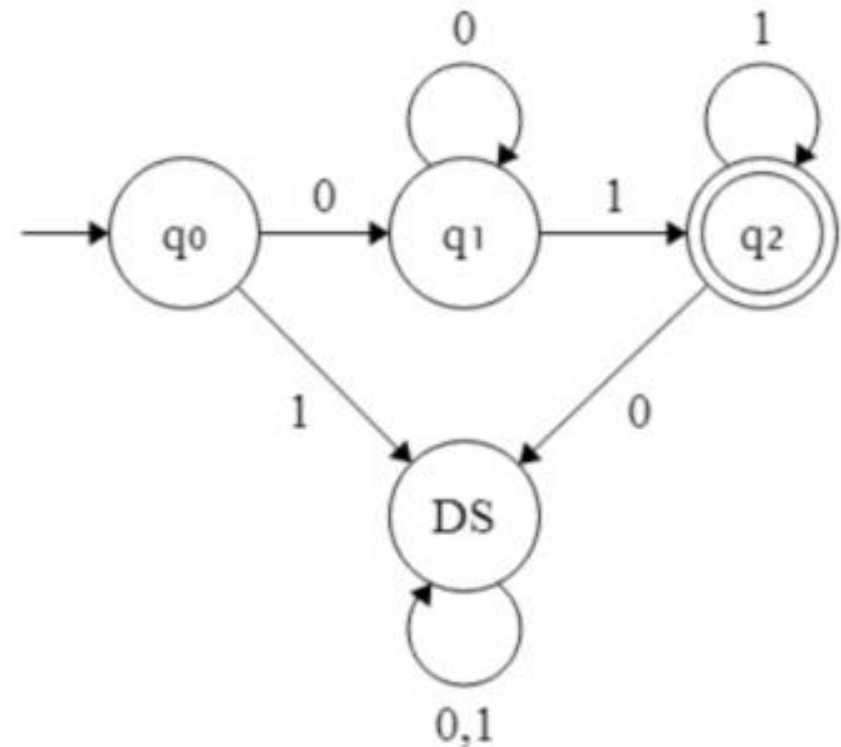


DFA Examples: Questions?

Language over alphabet $\{0,1\}$: The language $\{ 0^n 1^k \mid n \geq 1 \text{ and } k \geq 1 \}$



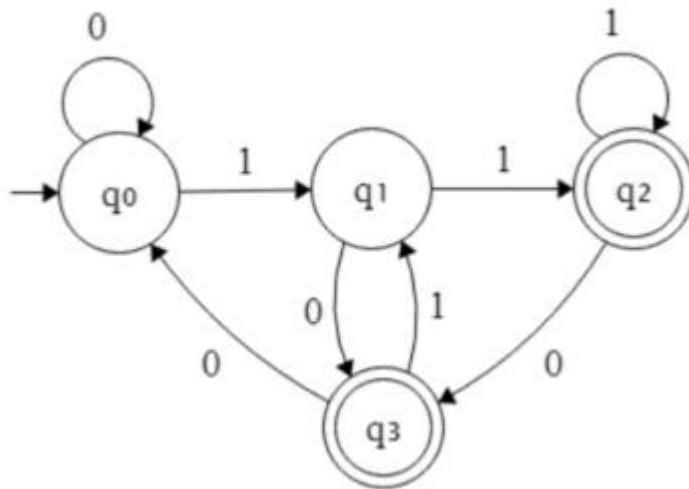
with DS drawn



DFA Examples: Questions?

- Give DFA's accepting the following languages over the alphabet $\{0,1\}$.

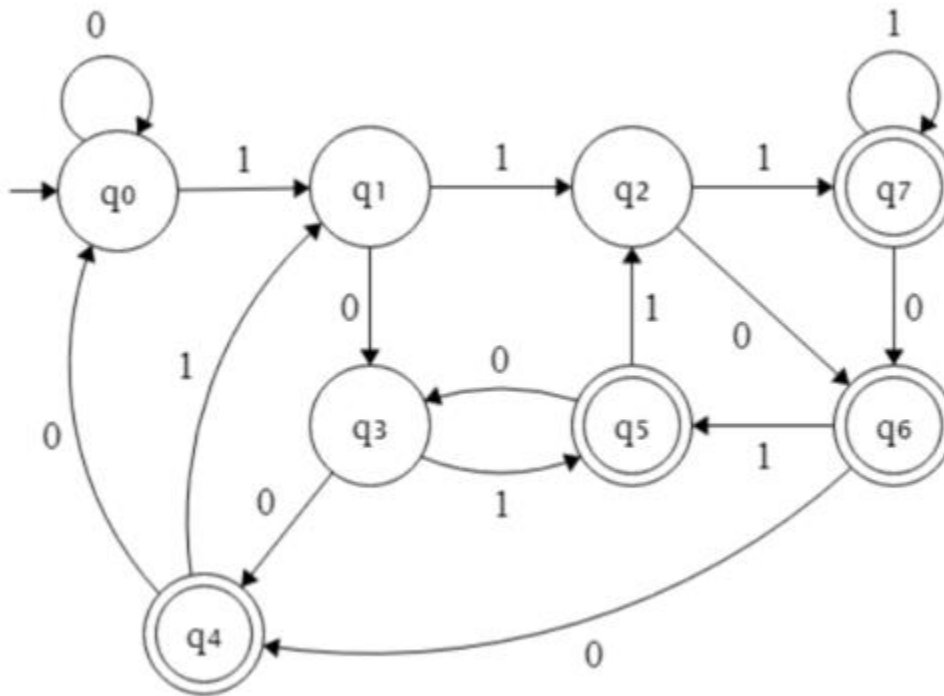
The strings whose second characters from the right end are 1.



DFA Examples: Questions?

- Give DFA's accepting the following languages over the alphabet $\{0,1\}$.

The strings whose third characters from the right end are 1.

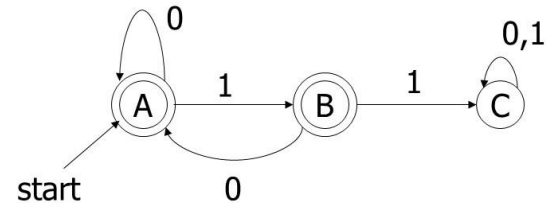


Proofs of Set Equivalence

- We need to prove that two descriptions of sets are in fact the same set. We want to prove that the language of a DFA is equal to a given set.

- Example:

- One set is the language of our example DFA
- The other one is “the set of strings of 0’s and 1’s with no consecutive 1’s”

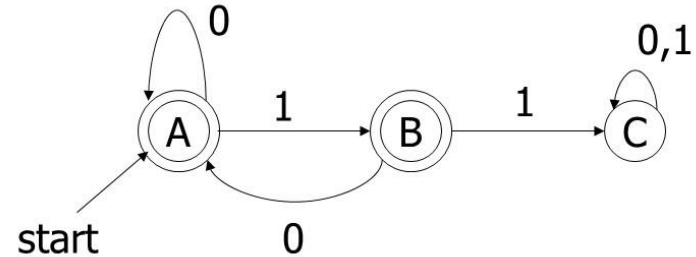


- In general, we want to prove sets S and T are equal (i.e. $S=T$).
- In order to prove $S=T$, we need to prove two parts:
 1. $S \subseteq T$ i.e. If w is in S, then w is in T.
 2. $T \subseteq S$ i.e. If w is in T, then w is in S.
- Example:
 - S = the language of our example DFA
 - T = “the set of strings of 0’s and 1’s with no consecutive 1’s”

Proofs of Set Equivalence

Proof Part 1 : $S \subseteq T$

- **To prove:** If w is accepted by our DFA then w has no consecutive 1's.
- The proof is an induction of length of w .



- **Important trick:** Expand the inductive hypothesis to be more detailed than we need.

Inductive Hypothesis:

1. If $\hat{\delta}(A, w) = A$, then w has no consecutive 1's and does not end in 1.
2. If $\hat{\delta}(A, w) = B$, then w has no consecutive 1's and ends in a single 1.

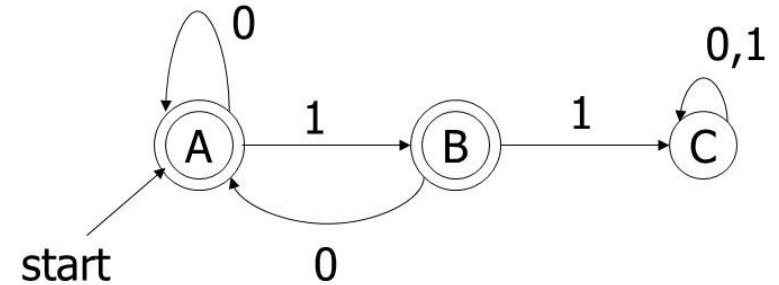
Proof Part 1 : $S \subseteq T$

Basis: $|w| = 0$; i.e., $w = \varepsilon$. $\hat{\delta}(A, w) = A$

- IH (1) holds since ε has no 1's at all.
- IH (2) holds *vacuously*, since $\hat{\delta}(A, \varepsilon)$ is not B.

Important concept:

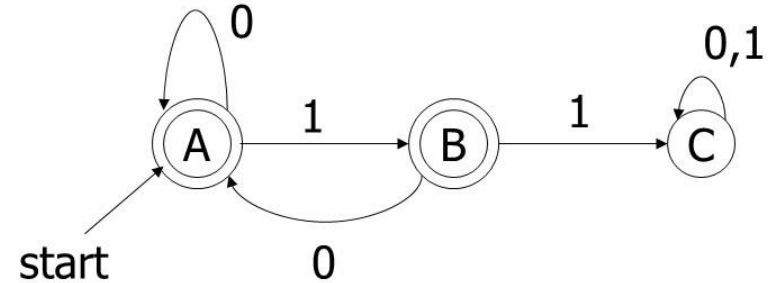
If the “if” part of “if..then” is false,
its conclusion is true.



Proof Part 1 : $S \subseteq T$

Inductive Step

- Need to prove IH (1) and IH (2) for $w = xa$.



Proof of IH (1): If $\hat{\delta}(A, w) = A$, then w has no consecutive 1's and does not end in 1.

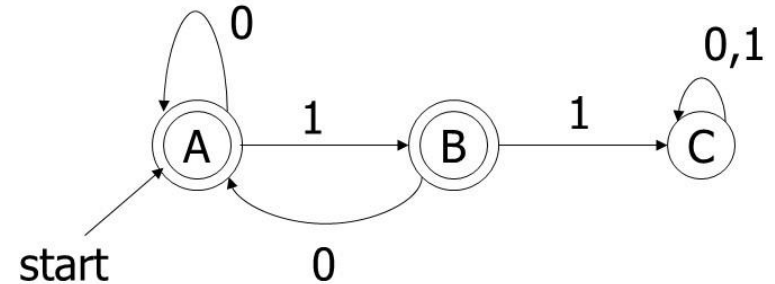
- Since $\hat{\delta}(A, w) = A$, $\hat{\delta}(A, x)$ must be A or B, and **a** must be 0 (look at the DFA).
- By the IH, x has no 11's.
- Thus, w has no 11's and does not end in 1.

Proof of IH (2): If $\hat{\delta}(A, w) = B$, then w has no consecutive 1's and ends in a single 1.

- Since $\hat{\delta}(A, w) = B$, $\hat{\delta}(A, x)$ must be A, and **a** must be 1 (look at the DFA).
- By the IH, x has no 11's and does not end in 1.
- Thus, w has no 11's and ends in a single 1.

Proof Part 2 : $T \subseteq S$

- **To prove:** If w has **no** 11's,
then w is accepted by our DFA.
- The proof is created using **contrapositive**.
- The **contrapositive** of “If w has **no** 11's, then w is accepted by our DFA” is
“If w is **not** accepted by our DFA then w has 11”.
- In general, the contrapositive of “if X then Y ” is the equivalent statement
“if **not** Y then **not** X .”

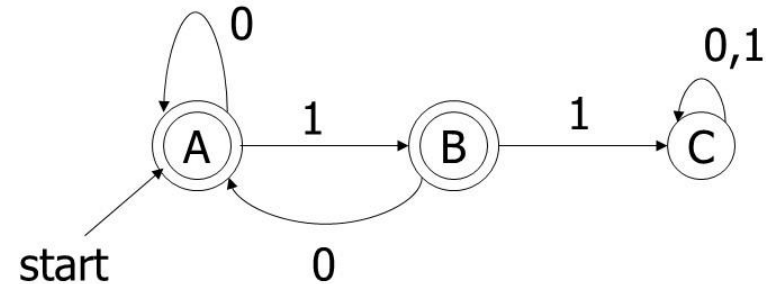


Proof Part 2 : $T \subseteq S$

Using Contrapositive

- Every w gets the DFA to exactly one state.
 - Simple inductive proof based on:
 - Every state has exactly one transition on 1, one transition on 0.
- The only way w is not accepted is if it gets to C.
- The only way to get to C [formally: $\hat{\delta}(A,w)=C$] is if $w=\mathbf{x1y}$, $\mathbf{x}$ gets to B, and $\mathbf{y}$ is the tail of w that follows what gets to C for the first time.
- If $\hat{\delta}(A,x)=B$ then surely $\mathbf{x=z1}$ for some $\mathbf{z}$.
- Thus, $w = \mathbf{z11y}$ and w has 11.
- By contrapositive,

If w has **no** 11's, then w is accepted by our DFA.



Regular Languages

- A language L is **regular** if it is the language **accepted by some DFA**.
 - A language is **regular** if it can be described by a **regular expression**.
- Some languages are **not regular**.
 - If a language is **not regular**, there is **no DFA for that language**.

Example 1:

- $L_1 = \{0^n 1^n \mid n \geq 1\}$ is not regular.
- The set of strings consisting of n 0's followed by n 1's, such that n is at least 1.
- Thus, $L_1 = \{01, 0011, 000111, \dots\}$

Example 2:

- $L_2 = \{w \mid w \text{ in } \{ (,) \}^* \text{ and } w \text{ is } \textit{balanced} \}$
 - Balanced parentheses are those that can appear in an arithmetic expression.
 - E.g.: $()$, $()()$, $(())$, $((()))$,...

DFA and Regular Languages

- Every DFA recognizes a regular language, and there is a DFA for every regular language.

DFA  **Regular Languages**

- Some languages are **not regular**. If a language is **not regular**, there is **no DFA** for that language.

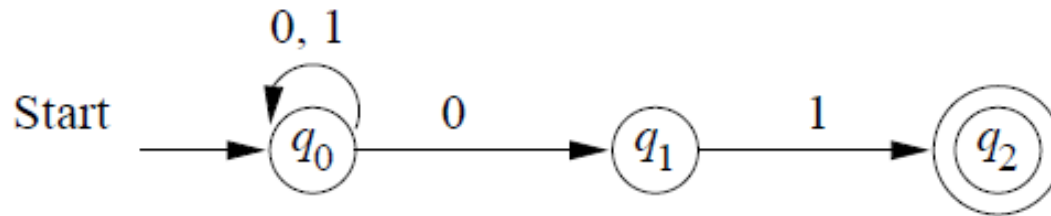
Non-Deterministic Finite Automaton (NFA)

Non-Deterministic Finite Automaton (NFA)

- A **nondeterministic finite automaton (NFA)** can be in several states at once, or it can "guess" which state to go to next.
- A NFA state can have more than one arc leaving from that state with a same symbol.
 - Transitions from a state on an input symbol can be to any set of states.
- A NFA can allow state-to-state transitions on ϵ input.
 - These transitions are done spontaneously, without looking at the input string.
- A NFA starts in the start state and it accepts if any sequence of choices for the string leads to a final state.
 - **Intuitively**: the NFA always “guesses right.”

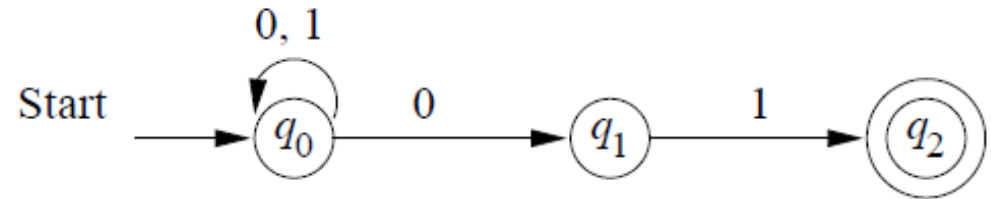
NFA – Example

- An automaton that accepts all and only strings ending in 01.

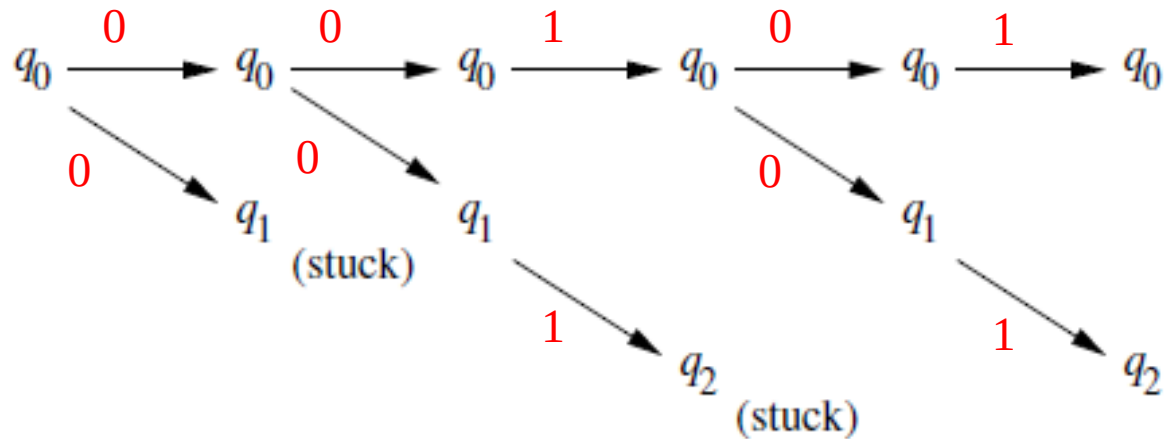


- State q_0 can go to q_0 or q_1 with the symbol 0. (non-determinism)
- NFA accepts a string w if there is a path accepts that string.
 - There can be other paths that do not accept that string.

NFA – Example



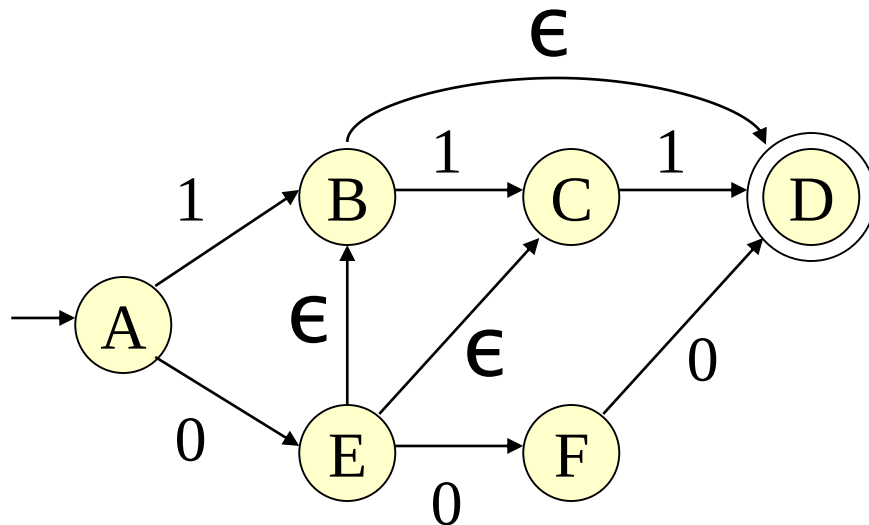
- What happens when the NFA processes the input 00101



- All missing arcs go to a death state, the death state goes to itself for all symbols, and the death state is a non-accepting state.

NFA – Example with ϵ transitions

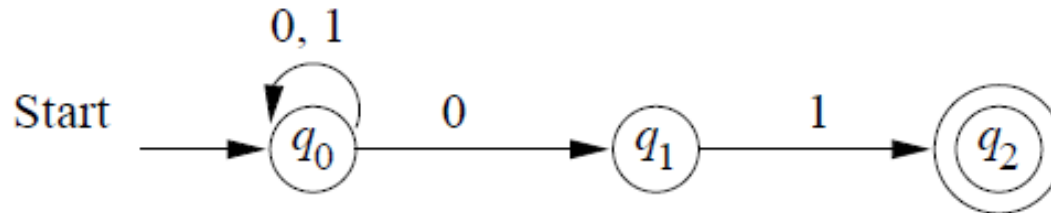
- This NFA accepts $\{1,111,0,011,01,000\}$
- This NFA can move from B to D without consuming a symbol.
 - It can also move from E to B without consuming a symbol.
 - It can also move from E to C without consuming a symbol.



Formal Definition of NFA

- A **Nondeterministic Finite Automaton (NFA)** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$
 1. Q is a finite set of states
 2. Σ is a finite set of symbols (alphabet)
 3. Delta (δ) is a transition function from $Q \times \Sigma \cup \{\epsilon\}$ to the power set of Q .
 4. q_0 is the start state ($q_0 \in Q$)
 5. F is a set of final (accepting) states ($F \subseteq Q$)
- Transition function takes two arguments: a state and an input symbol or ϵ .
- $\delta(q,a)$ = the set of the states that the DFA goes to when it is in state q and a is received.
 - where a is an input symbol or ϵ .

NFA – Table Representation



- The table representation of this NFA is as follows.

$\{ \{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\} \}$

- Its transition function δ is

	0	1	ϵ
$\rightarrow q_0$	$\{ q_0, q_1 \}$	$\{ q_0 \}$	ϕ
q_1	ϕ	$\{ q_2 \}$	ϕ
$* q_2$	ϕ	ϕ	ϕ

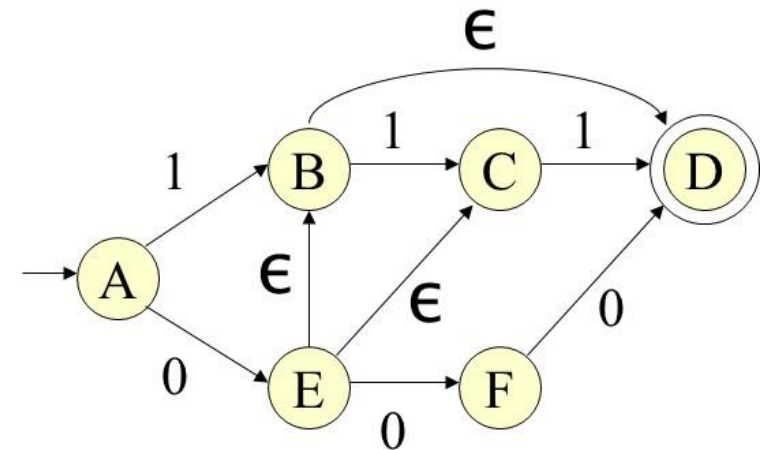
NFA – Table Representation

- The table representation of this NFA is as follows.

$\{ \{A,B,C,D,E,F\}, \{0,1\}, \delta, A, \{D\} \}$

- Its transition function δ is

	0	1	ϵ
$\rightarrow$ A	{ E }	{ B }	ϕ
B	ϕ	{ C }	{ D }
C	ϕ	{ D }	ϕ
* D	ϕ	ϕ	ϕ
E	{ F }	ϕ	{ B, C }
F	{ D }	ϕ	ϕ



Epsilon Closure

- We **close** a state by **adding all states reachable by a sequence $\varepsilon\varepsilon\dots\varepsilon$** .
- **ECLOSE(q)** is the epsilon closure of the state q.

Inductive definition of ECLOSE(q):

Basis: $q \in \text{ECLOSE}(q)$

Induction: If $p \in \text{ECLOSE}(q)$ and $r \in \delta(p, \varepsilon)$, then $r \in \text{ECLOSE}(q)$

Epsilon Closure

$ECLOSE(1) = \{1,2,3,4,6\}$

$ECLOSE(2) = \{2,3,6\}$

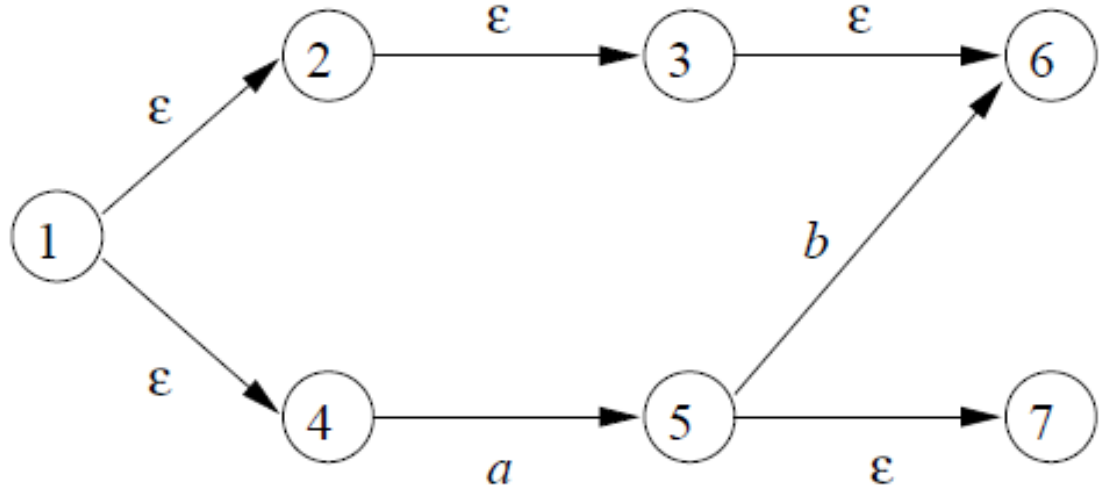
$ECLOSE(3) = \{3,6\}$

$ECLOSE(4) = \{4\}$

$ECLOSE(5) = \{5,7\}$

$ECLOSE(6) = \{6\}$

$ECLOSE(7) = \{7\}$



Epsilon Closure of Set of States

- **ECLOSE(S)** is the epsilon closure of the set of states S.

$$\mathbf{ECLOSE(S)} = \bigcup_{q \in S} \mathbf{ECLOSE(q)}$$

Extended Delta Function for NFA – Delta Hat $\hat{\delta}$

- The transition function δ can be extended to **extended delta function** $\hat{\delta}$ that operates on **states** and **strings** (as opposed to states and symbols).

Inductive definition of extended delta function $\hat{\delta}$ for NFA:

Basis: $\hat{\delta}(q, \epsilon) = \text{ECLOSE}(q)$

Induction:

$$\hat{\delta}(q, xa) = \bigcup_{p \in \bigcup_{r \in \hat{\delta}(q, x)} \delta(r, a)} \text{ECLOSE}(p)$$

Acceptance in an NFA

- An NFA $(Q, \Sigma, \delta, q_0, F)$ accepts a string w in Σ^* iff we can write $w=y_1y_2\dots y_m$ where each $y_i \in \Sigma \cup \{\epsilon\}$ and there is a sequence of states $s_0, \dots, s_m \in Q$ such that
 1. $s_0 = q_0$
 2. $s_{i+1} \in \delta(s_i, y_{i+1})$ for each $i = 0, \dots, m-1$
 3. $s_m \in F$

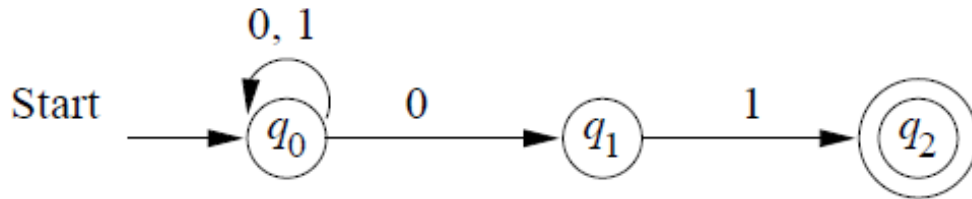
Language of a NFA

- The language accepted by an NFA A is

$$L(A) = \{ w \mid \hat{\delta}(q_0, w) \cap F \neq \phi \}$$

- i.e. a string w is accepted by a NFA A iff the states that are reachable from the starting state by consuming w contain at least one final state.

NFA – $\hat{\delta}$ Acceptance Example



	0	1	ϵ
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$	$\emptyset$
$* q_2$	$\emptyset$	$\emptyset$	$\emptyset$

- Let us use $\hat{\delta}$ when the NFA processes the input 00101

- $\hat{\delta}(q_0, \epsilon) = \text{ECLOSE}(q_0) = \{q_0\}$
- $\hat{\delta}(q_0, \mathbf{0}) = \text{ECLOSE}(q_0) \cup \text{ECLOSE}(q_1) = \{q_0, q_1\}$
- $\hat{\delta}(q_0, \mathbf{00}) = \text{ECLOSE}(q_0) \cup \text{ECLOSE}(q_1) = \{q_0, q_1\}$
- $\hat{\delta}(q_0, \mathbf{001}) = \text{ECLOSE}(q_0) \cup \text{ECLOSE}(q_2) = \{q_0, q_2\}$
- $\hat{\delta}(q_0, \mathbf{0010}) = \text{ECLOSE}(q_0) \cup \text{ECLOSE}(q_1) = \{q_0, q_1\}$
- $\hat{\delta}(q_0, \mathbf{00101}) = \text{ECLOSE}(q_0) \cup \text{ECLOSE}(q_2) = \{q_0, q_2\}$

where $\delta(q_0, \mathbf{0}) = \{q_0, q_1\}$

where $\delta(q_0, \mathbf{0}) \cup \delta(q_1, \mathbf{0}) = \{q_0, q_1\} \cup \{\} = \{q_0, q_1\}$

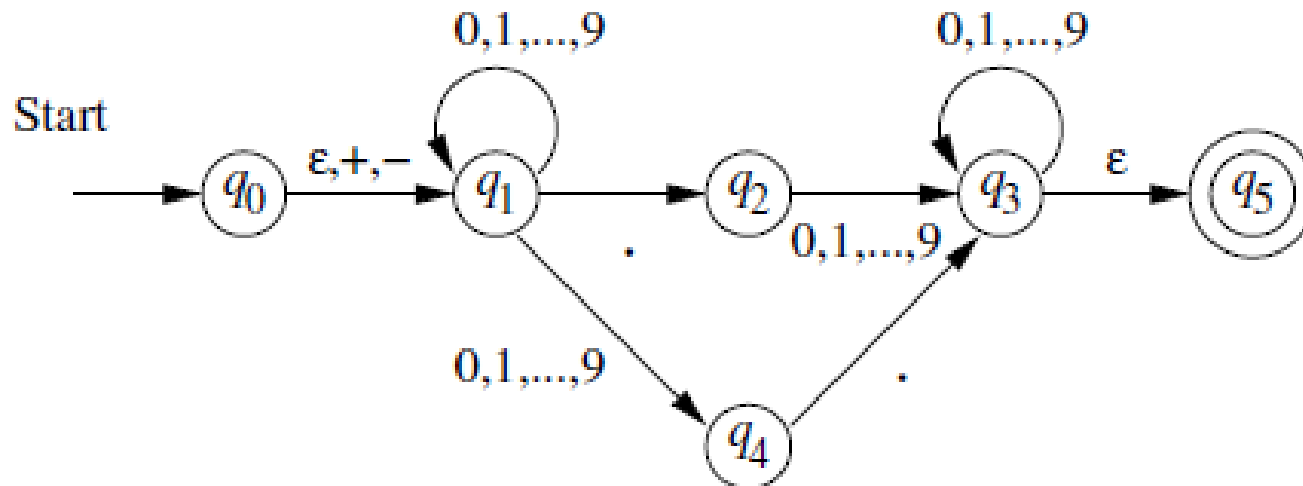
where $\delta(q_0, \mathbf{1}) \cup \delta(q_1, \mathbf{1}) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$

where $\delta(q_0, \mathbf{0}) \cup \delta(q_2, \mathbf{0}) = \{q_0, q_1\} \cup \{\} = \{q_0, q_1\}$

where $\delta(q_0, \mathbf{1}) \cup \delta(q_1, \mathbf{1}) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$

NFA - Example

- An NFA accepting decimal numbers consisting of:
 1. an optional + or - sign
 2. a string of digits
 3. a decimal point
 4. another string of digits
- One of the strings in (2) and (4) are optional.



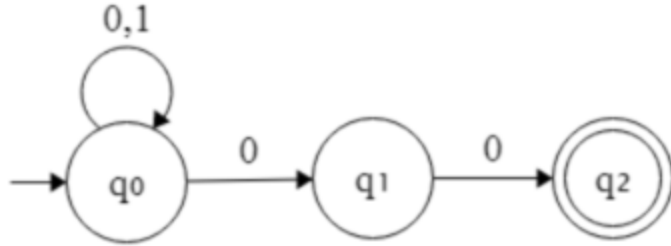
NFA Examples: Questions?

- Give NFA's accepting the following languages over the alphabet $\{0,1\}$.
 1. The set of all strings ending in 00.
 2. The set of all strings ending in 1010.
 3. The strings whose second characters from the right end are 1.
 4. The strings whose third characters from the right end are 1.

NFA Examples: Questions?

- Give NFA's accepting the following languages over the alphabet $\{0,1\}$.

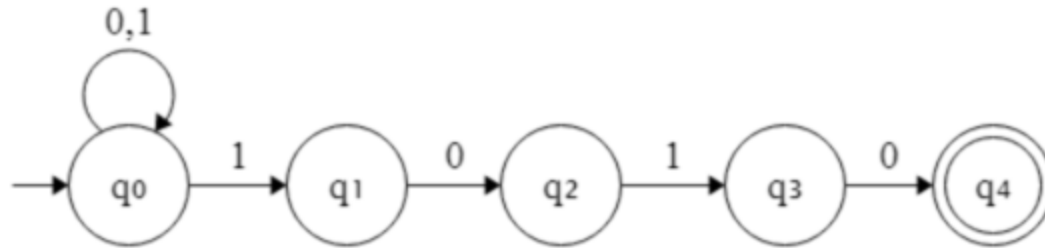
The set of all strings ending in 00.



NFA Examples: Questions?

- Give NFA's accepting the following languages over the alphabet $\{0,1\}$.

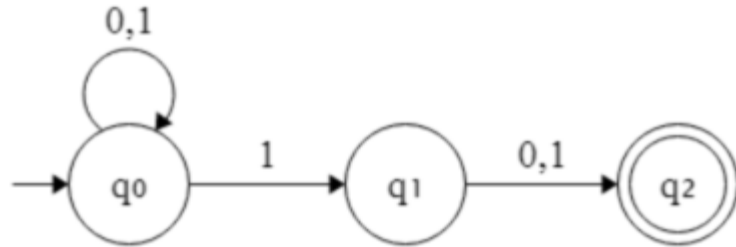
The set of all strings ending in 1010.



NFA Examples: Questions?

- Give NFA's accepting the following languages over the alphabet $\{0,1\}$.

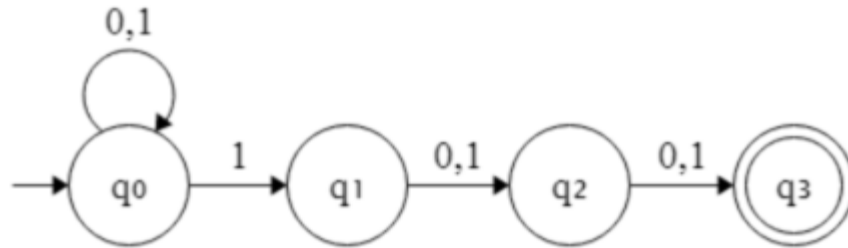
The strings whose second characters from the right end are 1.



NFA Examples: Questions?

- Give NFA's accepting the following languages over the alphabet $\{0,1\}$.

The strings whose third characters from the right end are 1.



Equivalence of DFA and NFA

Equivalence of DFA and NFA

- NFA's are usually easier to construct.
- Surprisingly, for any NFA N there is a DFA D , such that $L(D) = L(N)$, and vice versa.
- This involves the **subset construction**.
- Given an NFA N

$$N = (Q_N, \Sigma, \delta_N, q_0, F_N)$$

we can construct a DFA D

$$D = (Q_D, \Sigma, \delta_D, q_D, F_D)$$

such that $L(D) = L(N)$

Equivalence of DFA and NFA

Subset Construction

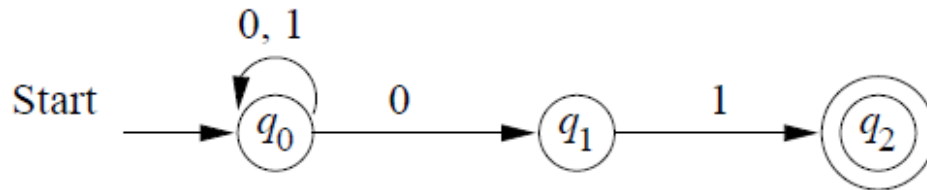
$$N = (Q_N, \Sigma, \delta_N, q_0, F_N)$$

$$D = (Q_D, \Sigma, \delta_D, q_D, F_D)$$

- $Q_D = \{ S \mid S \subseteq Q_N \}$
 - Note that $|Q_D| = 2^{|Q_N|}$ although most states are likely to be garbage.
- $q_D = \text{ECLOSE}(q_0)$
- $F_D = \{ S \subseteq Q_N \mid S \cap F_N \neq \emptyset \}$
- For every $S \subseteq Q_N$ and $a \in \Sigma$

$$\delta_D(S, a) = \bigcup_{p \in S} \text{ECLOSE}(\delta_N(p, a))$$

Subset Construction - Example



$$q_D = \text{ECLOSE}(q_0) = \{q_0\}$$

$$Q_D = \{ \{\}, \{q_0\}, \{q_1\}, \{q_2\}, \\ \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \\ \{q_0, q_1, q_2\} \}$$

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\star\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\star\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

But, some of the states are **NOT accessible** from the starting state $q_D = \{q_0\}$.

Subset Construction – Accessible States

- We can often avoid the exponential blow-up by constructing the transition table for D only for accessible states S as follows:

Basis: $S = \text{ECLOSE}(q_0)$ is accessible in D

Induction:

If state S is accessible, so are states in $\bigcup_{a \in \Sigma} \text{ECLOSE}(\delta_D(S, a))$

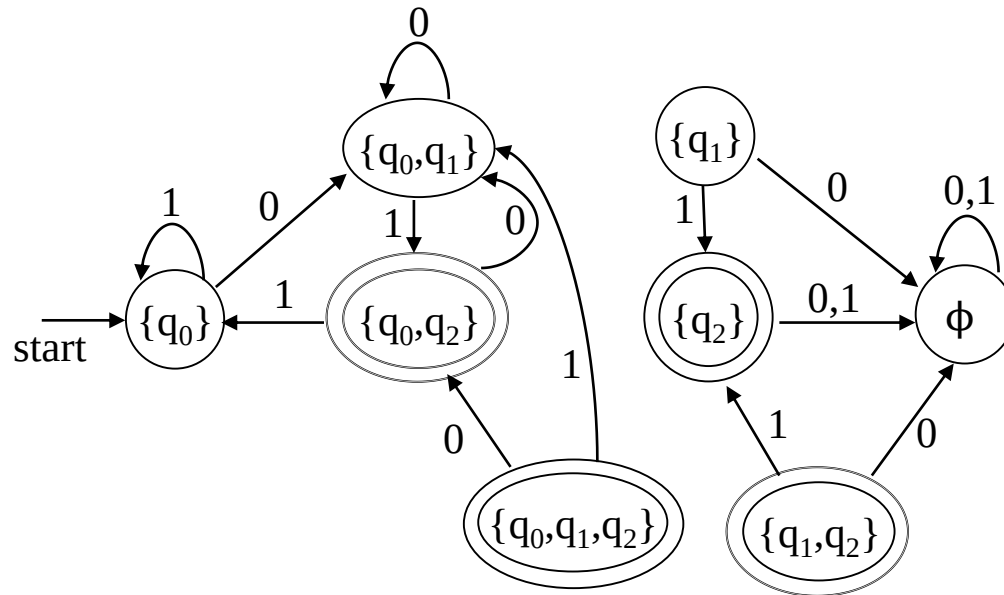
Subset Construction – Accessible States

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\star\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\star\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

Accessible States:

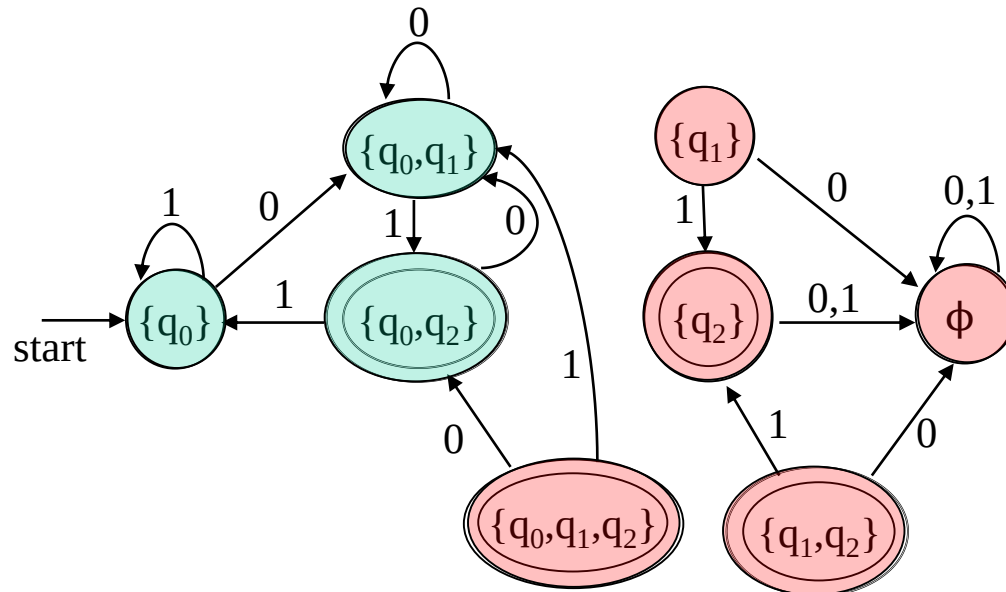
- **Basis:** $\{q_0\}$
- Since $\{q_0\}$ is accessible, $\{q_0, q_1\}$ is accessible.
- Since $\{q_0, q_1\}$ is accessible, $\{q_0, q_2\}$ is accessible.
- There are **NO** more accessible states.
- Thus all accessible states (states of DFA) are $\{q_0\}, \{q_0, q_1\}, \{q_0, q_2\}$

Subset Construction – Accessible States



	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\star\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\star\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

Subset Construction – Accessible States



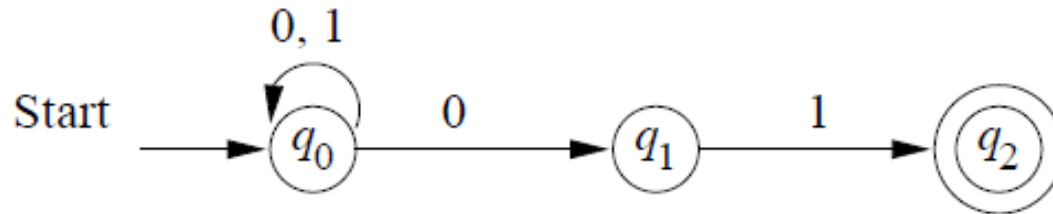
accessible states are
 $\{q_0\}$, $\{q_0, q_1\}$, $\{q_0, q_2\}$

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star \{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\star \{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\star \{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star \{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

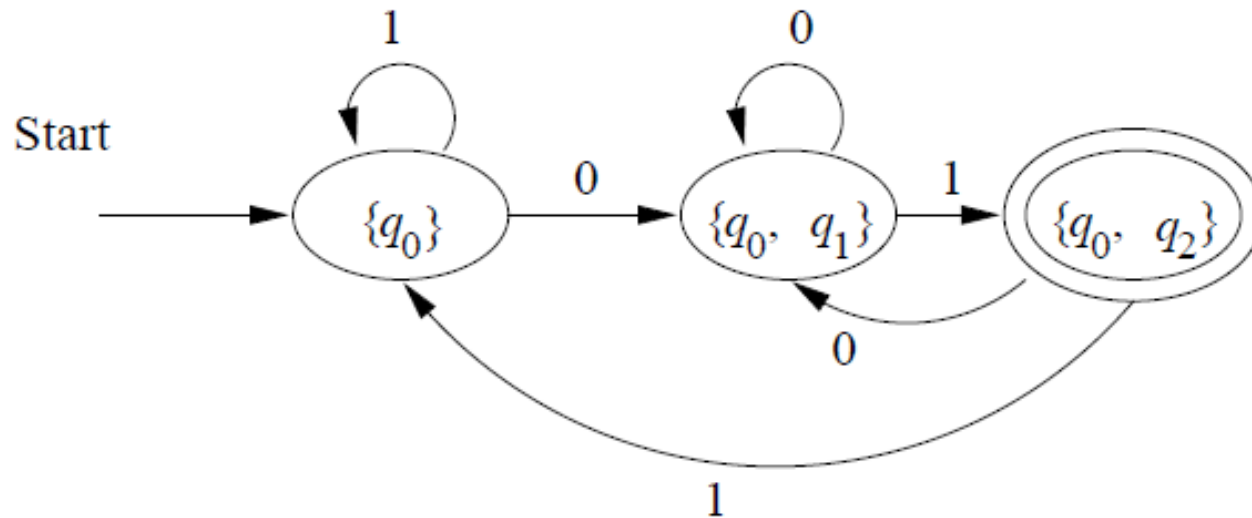
Non-accessible states are $\{q_1\}$,
 $\{q_2\}$, $\{\}$, $\{q_1, q_2\}$, $\{q_0, q_1, q_2\}$

Subset Construction – Accessible States (example)

NFA



DFA



Equivalence of DFA and NFA – Theorem 1

Theorem: Let D be the subset construction DFA of an NFA N . Then $L(D) = L(N)$.

Proof: We show on an induction on $|w|$ that

$$\widehat{\delta}_D(\text{ECLOSE}(q_0), w) = \widehat{\delta}_N(q_0, w)$$

Basis: $w = \varepsilon$. The claim follows from definition.

Induction:

$$\begin{aligned}\widehat{\delta}_D(\text{ECLOSE}(q_0), xa) &= \delta_D(\widehat{\delta}_D(\text{ECLOSE}(q_0), x), a) && \text{by definition of } \widehat{\delta}_D \\ &= \delta_D(\widehat{\delta}_N(q_0, x), a) && \text{by IH} \\ &= \bigcup_{p \in \widehat{\delta}_N(q_0, x)} \text{ECLOSE}(\delta_N(p, a)) && \text{by construction of } \delta_D \\ &= \widehat{\delta}_N(q_0, xa) && \text{by definition of } \widehat{\delta}_N\end{aligned}$$

Equivalence of DFA and NFA – Theorem 2

Theorem: A language L is accepted by some DFA if and only if L is accepted by some NFA.

Proof:

if-part: The **if-part** is proved by the previous theorem (Theorem 1).

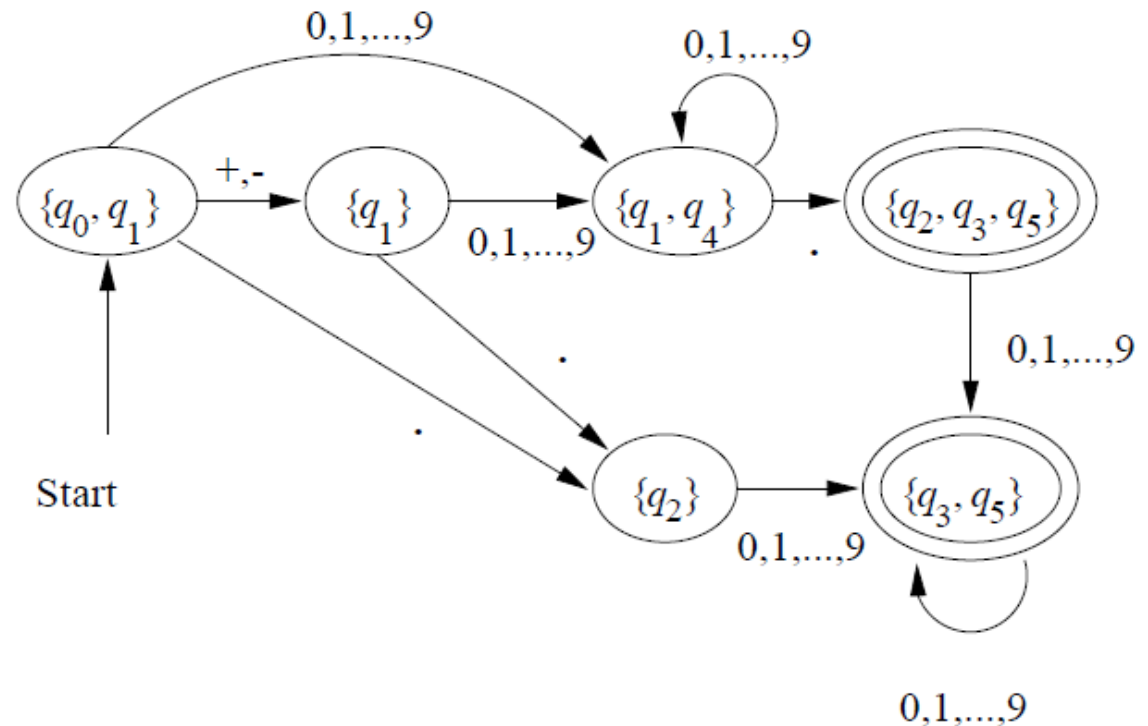
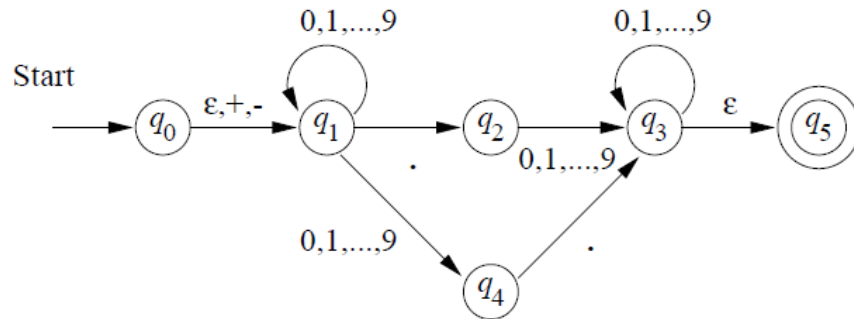
only-if-part

- For the **only-if-part**, we note that any DFA can be converted to an equivalent NFA by modifying the δ_D to δ_N by the rule:
 - If $\delta_D(q,a)=p$, then to $\delta_N(q,a)=\{p\}$.
 - The rest of NFA will be same as DFA.
- By induction on $|w|$, it can be shown that

$$\text{If } \hat{\delta}_D(q_0, w) = p, \text{ then } \hat{\delta}_N(q_0, w) = \{p\}$$

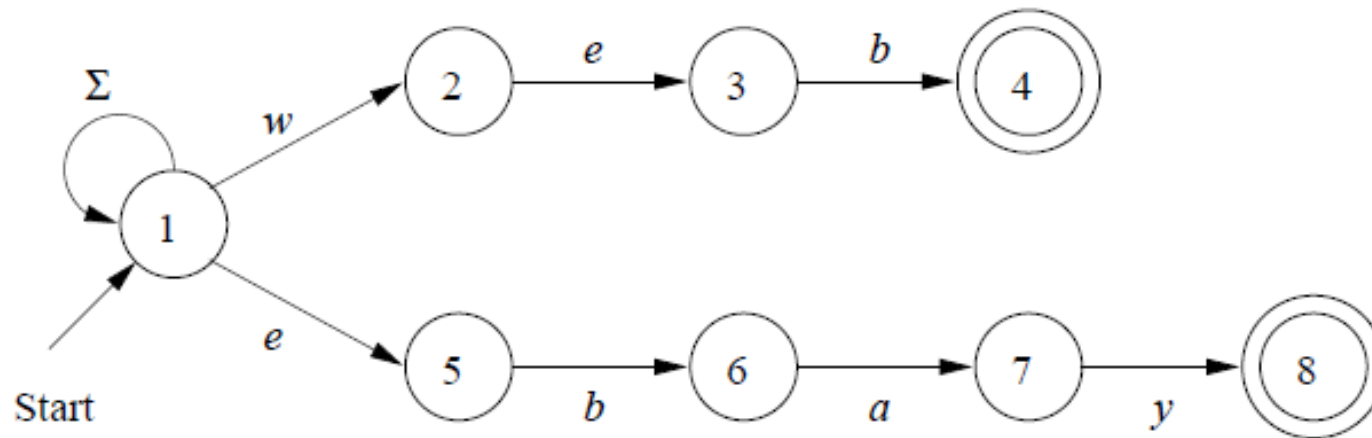
Equivalence of DFA and NFA

Subset Construction - Example

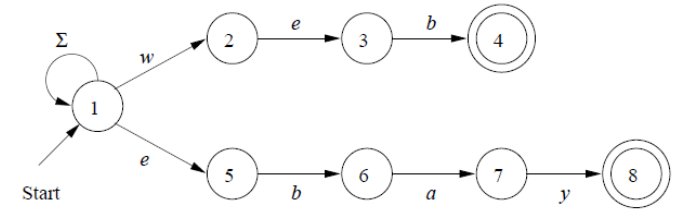
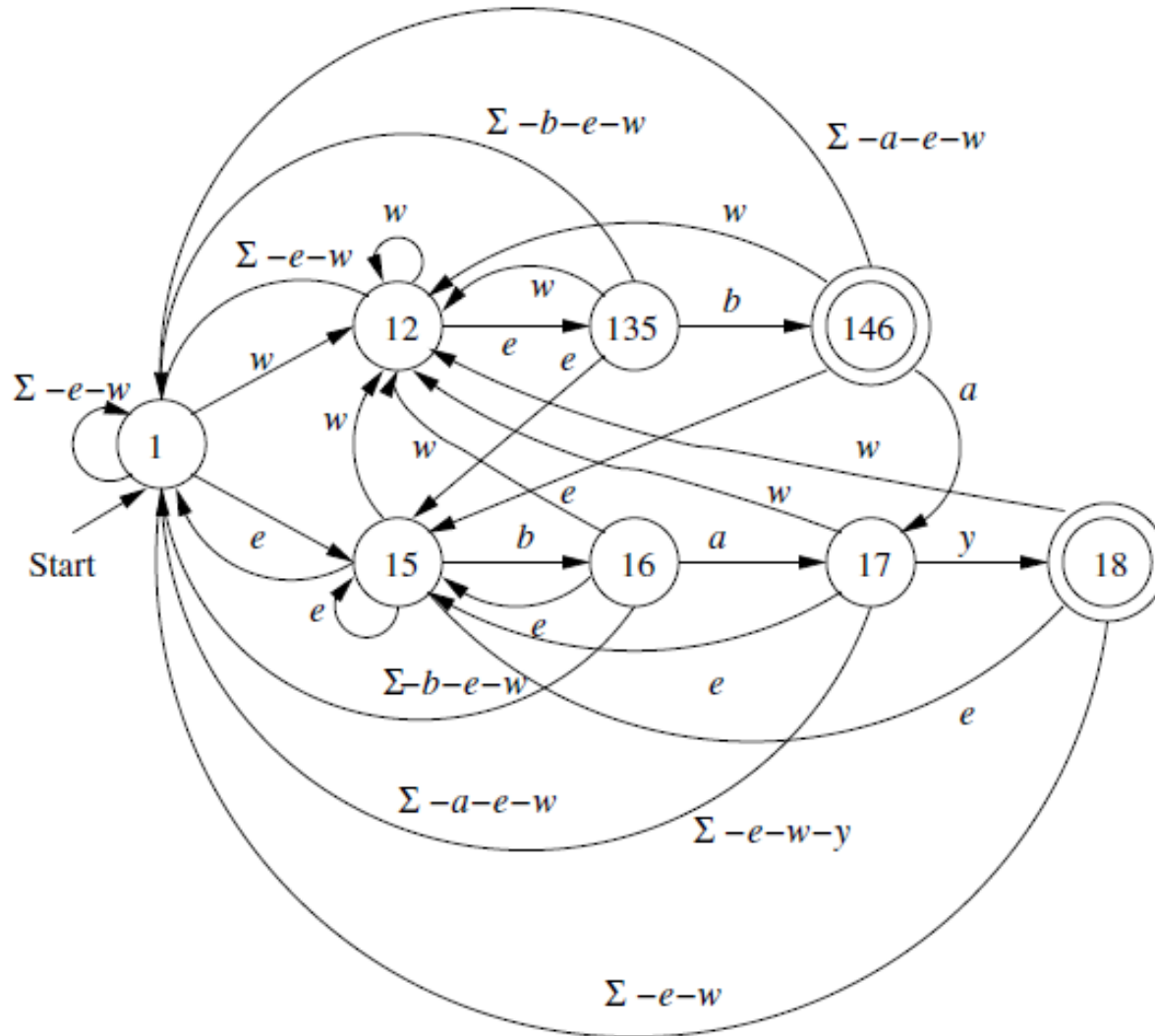


NFA for Text Search

- An NFA accepting the set of words ending with **ebay** or **web**

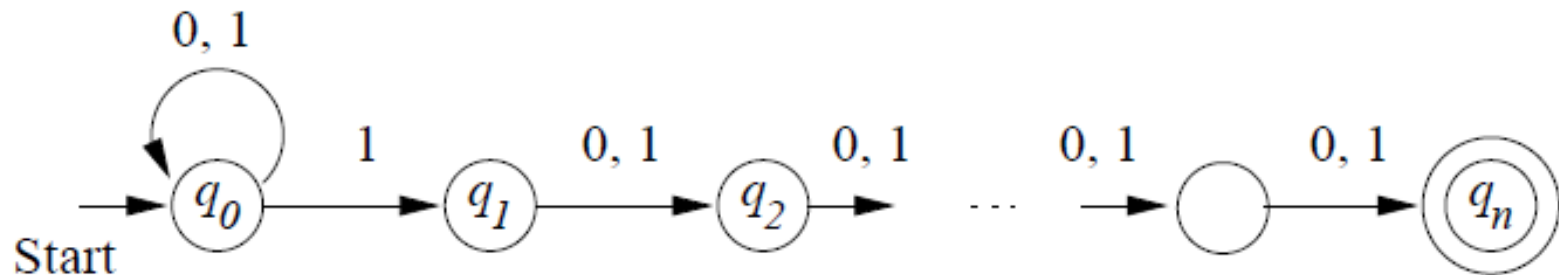


Corresponding DFA for Text Search



A Bad Case for Subset Construction - Exponential Blow-Up

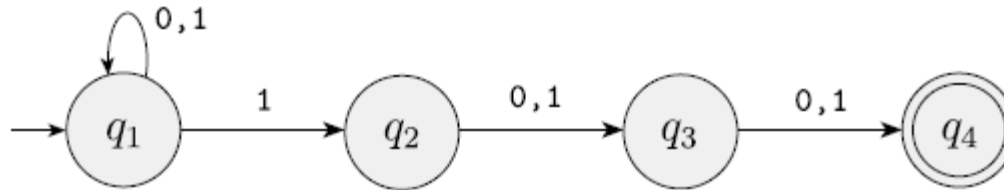
- There is an NFA N with $n+1$ states that has no equivalent DFA with fewer than 2^n states



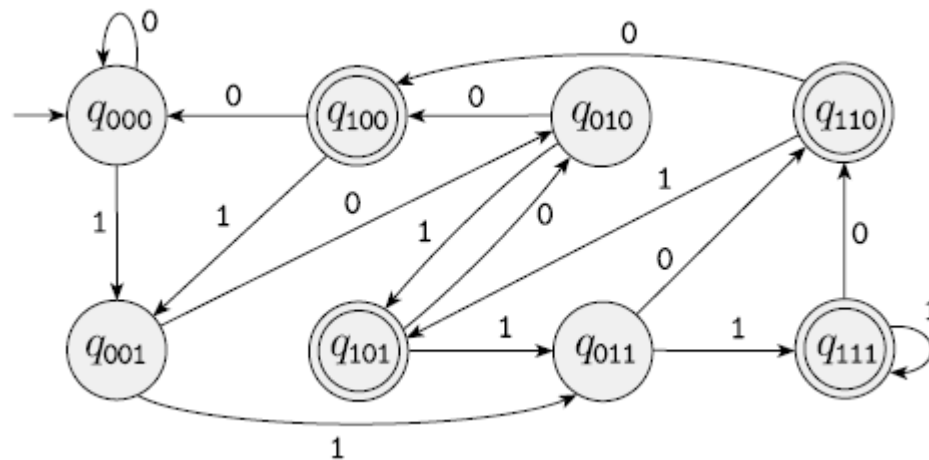
$$L(N) = \{x1c_2c_3 \cdots c_n : x \in \{0, 1\}^*, c_i \in \{0, 1\}\}$$

A Bad Case for Subset Construction - Exponential Blow-Up

- A NFA which recognizes the strings whose third characters from the right end are 1.

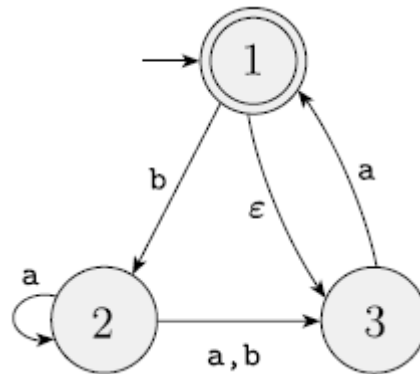


- An equivalent DFA which recognizes the strings whose third characters from the right end are 1.



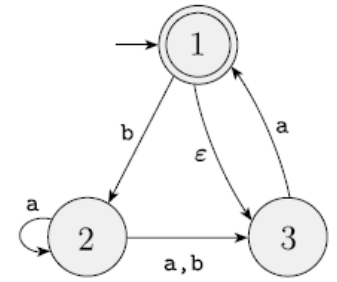
Equivalence of DFA and NFA: Questions?

- Construct a DFA that is equivalent to the following NFA.



Equivalence of DFA and NFA: Questions?

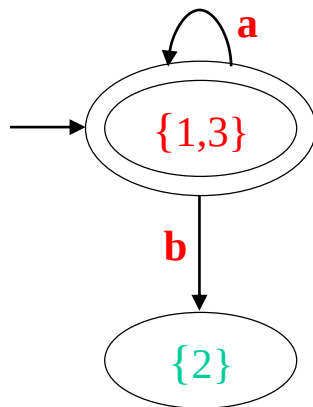
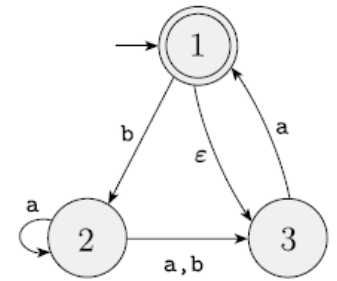
$$\text{ECLOSE}(1) = \{1, 3\}$$



Equivalence of DFA and NFA: Questions?

$$\delta_D(\{1,3\},a) = \text{ECLOSE}(\delta_N(1,a)) \cup \text{ECLOSE}(\delta_N(3,a)) = \{\} \cup \{1,3\} = \{1,3\}$$

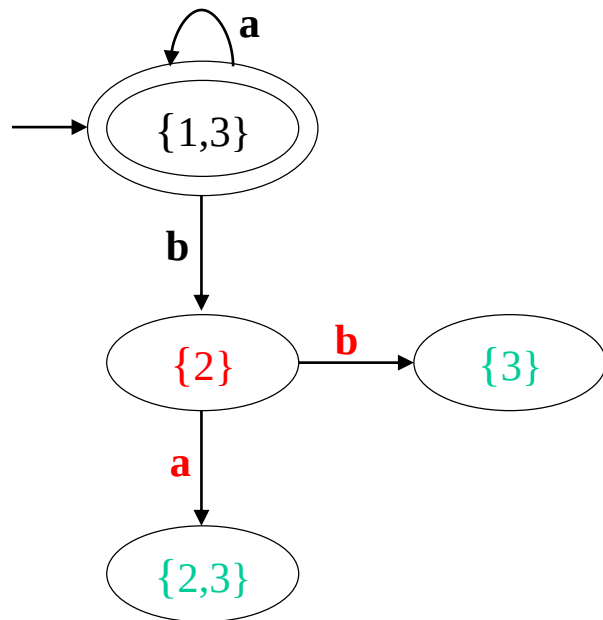
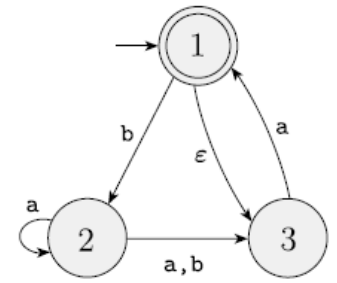
$$\delta_D(\{1,3\},b) = \text{ECLOSE}(\delta_N(1,b)) \cup \text{ECLOSE}(\delta_N(3,b)) = \{2\} \cup \{\} = \{2\}$$



Equivalence of DFA and NFA: Questions?

$$\delta_D(\{2\}, a) = \text{ECLOSE}(\delta_N(2, a)) = \{2, 3\}$$

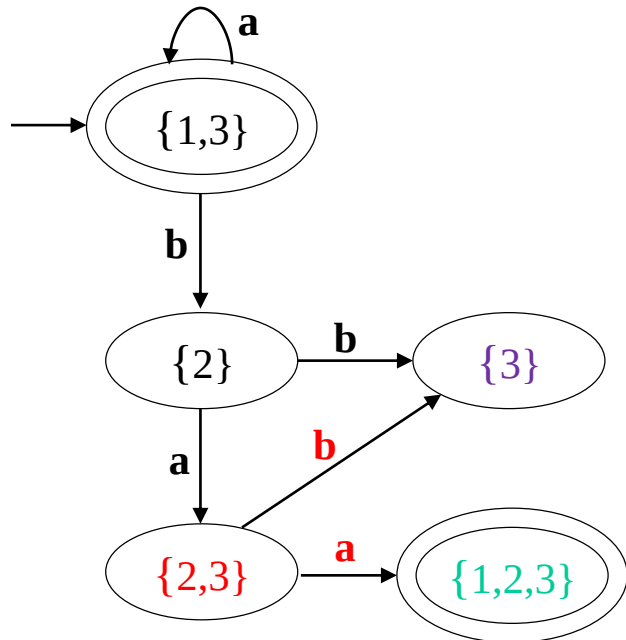
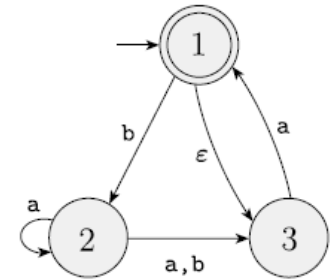
$$\delta_D(\{2\}, b) = \text{ECLOSE}(\delta_N(2, b)) = \{3\}$$



Equivalence of DFA and NFA: Questions?

$$\delta_D(\{2,3\},a) = \text{ECLOSE}(\delta_N(2,a)) \cup \text{ECLOSE}(\delta_N(3,a)) = \{2,3\} \cup \{1,3\} = \{1,2,3\}$$

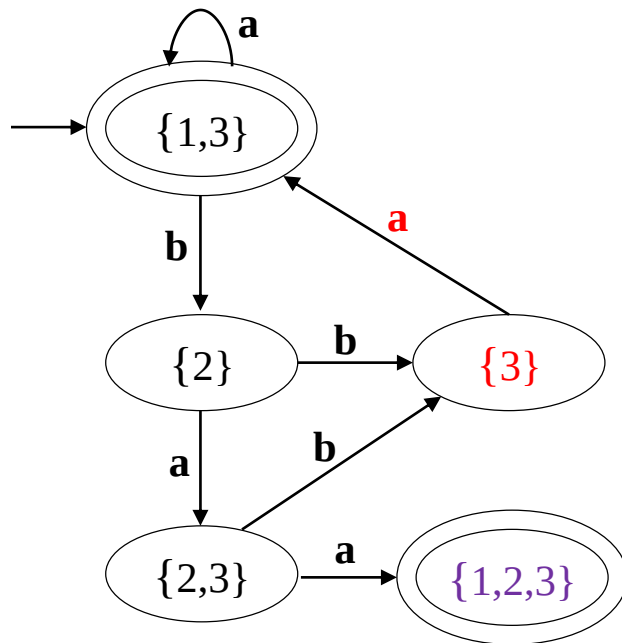
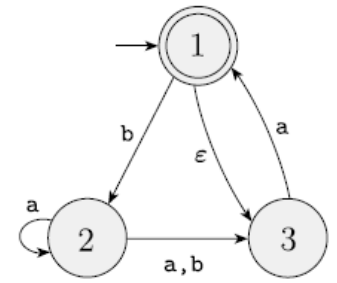
$$\delta_D(\{2,3\},b) = \text{ECLOSE}(\delta_N(2,b)) \cup \text{ECLOSE}(\delta_N(3,b)) = \{3\} \cup \{\} = \{3\}$$



Equivalence of DFA and NFA: Questions?

$$\delta_D(\{3\}, a) = \text{ECLOSE}(\delta_N(3, a)) = \{1, 3\}$$

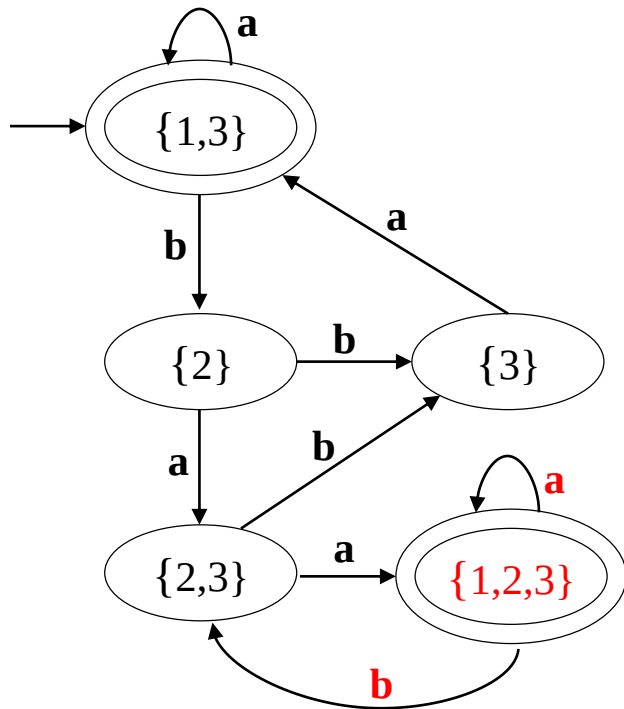
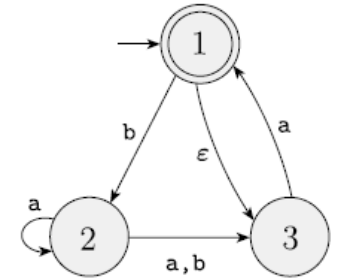
$$\delta_D(\{3\}, b) = \text{ECLOSE}(\delta_N(3, b)) = \{3\}$$



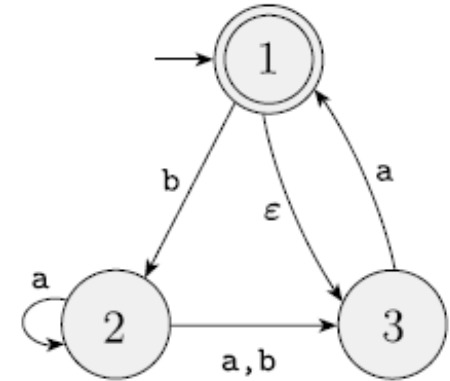
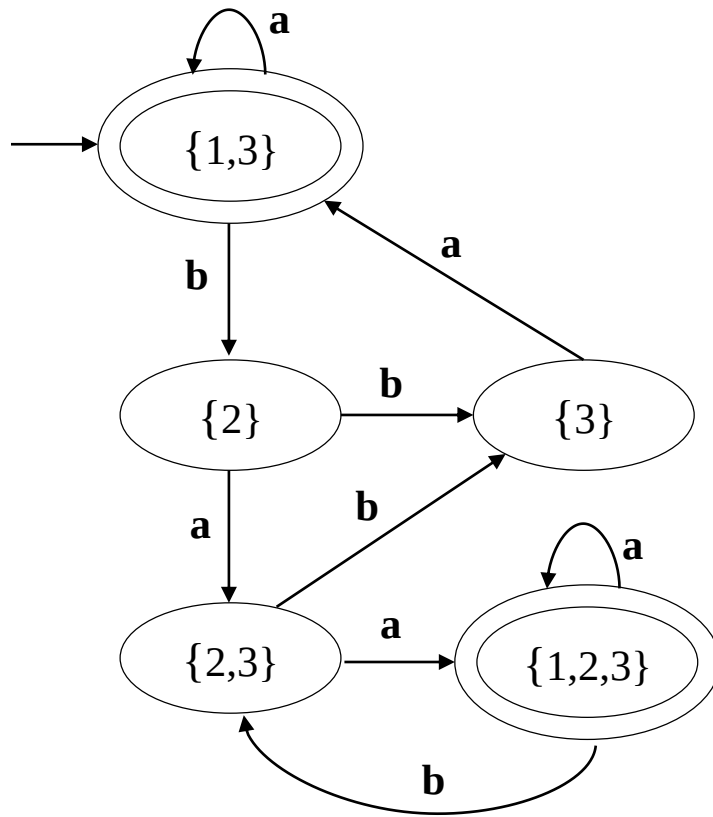
Equivalence of DFA and NFA: Questions?

$$\begin{aligned}\delta_D(\{1,2,3\},a) &= \text{ECLOSE}(\delta_N(1,a)) \cup \text{ECLOSE}(\delta_N(2,a)) \cup \text{ECLOSE}(\delta_N(3,a)) \\ &= \{\} \cup \{2,3\} \cup \{1,3\} = \{1,2,3\}\end{aligned}$$

$$\begin{aligned}\delta_D(\{1,2,3\},b) &= \text{ECLOSE}(\delta_N(1,b)) \cup \text{ECLOSE}(\delta_N(2,b)) \cup \text{ECLOSE}(\delta_N(3,b)) \\ &= \{2\} \cup \{3\} \cup \{\} = \{2,3\}\end{aligned}$$



Equivalence of DFA and NFA: Questions?



Equivalence of DFA and NFA: Summary

- Every DFA recognizes a regular language, and there is a DFA for every regular language.
- There is an equivalent DFA (their languages are equal) for every NFA, and there is an equivalent NFA for every DFA.
- Thus, every NFA recognizes a regular language, and there is a NFA for every regular language.

